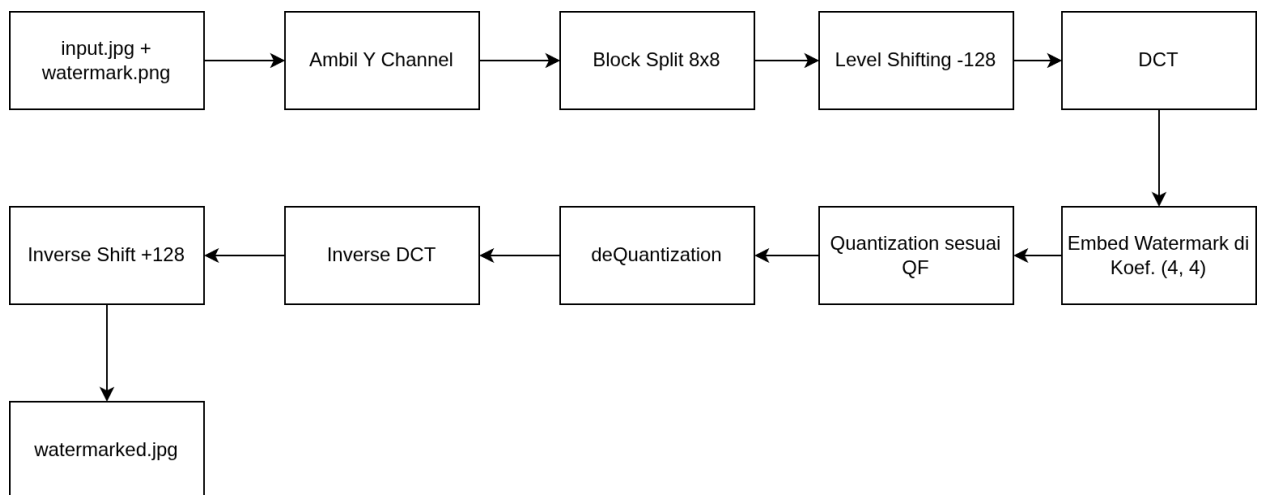


Watermarking

Repository: <https://github.com/EmirMF/sismul-watermark>

a. Penjelasan Proses Watermarking



1. Input & Ambil Y Channel

Pada tahap awal, program akan membaca dua file

```
host = cv2.imread(host_path, cv2.IMREAD_COLOR)
watermark = cv2.imread(watermark_path, cv2.IMREAD_GRAYSCALE)
```

input.jpg adalah gambar utama yang akan disisipkan watermark. watermark.png adalah citra watermark biner, misalnya logo hitam-putih. Gambar asli dibaca sebagai BGR/color, sedangkan watermark dibaca grayscale karena nantinya hanya dibutuhkan nilai gelap/terang sebagai bit 0 dan 1. Watermark kemudian diubah ke dalam bentuk biner

```
_, binary = cv2.threshold( watermark.astype(np.uint8), 0, 1,
cv2.THRESH_BINARY + cv2.THRESH_OTSU )
```

Dari gambar asli, diambil Y channelnya dari YCrCb, karena JPEG lebih banyak bekerja pada komponen luminance tersebut. Channel Cb dan Cr akan disimpan dan nantinya akan digabungkan kembali dengan hasil akhir

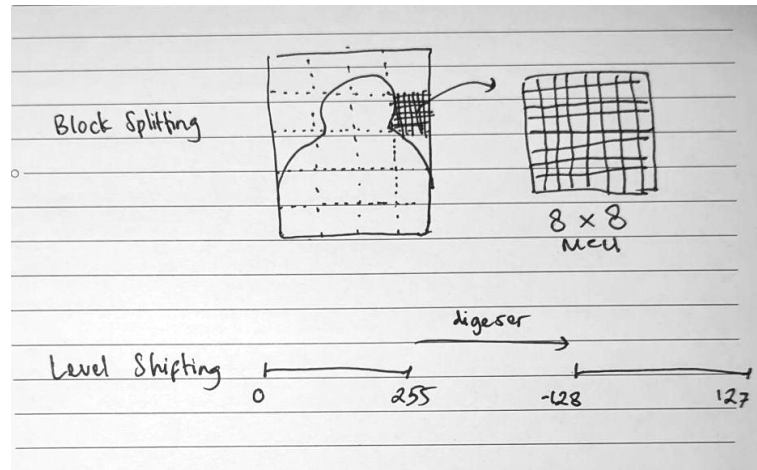
```

ycrcb = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)
return ycrcb[:, :, 0].astype(np.uint8), ycrcb[:, :, 1:].copy()

```

Watermark.png juga dibaca oleh program sebagai grayscale dan kemudian diubah menjadi citra biner, yang pixel gelap menjadi 0 dan pixel terang menjadi 1

2. Block Split 8x8 & Level Shifting



Proses kompresi JPG bekerja dengan menggunakan blok 8x8, sehingga gambar asli tadi dipotong potong menjadi blok-blok berukuran 8x8 pixel.

```

cropped_height = height - (height % 8)
cropped_width = width - (width % 8)
return image[:cropped_height, :cropped_width].copy()

```

Untuk pemrosesan blok dilakukan di loop ini

```

for row in range(0, luminance.shape[0], 8):
    for col in range(0, luminance.shape[1], 8):
        block = luminance[row : row + 8, col : col + 8]

```

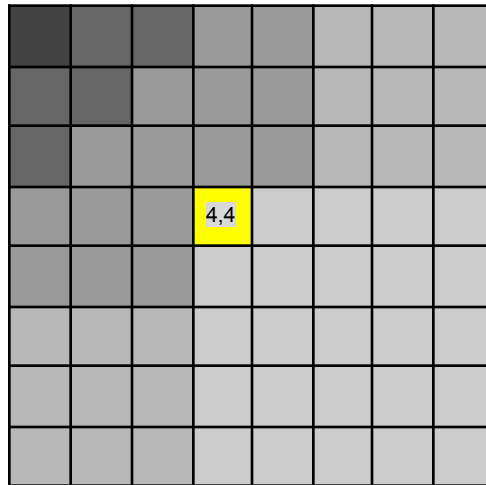
Kemudian untuk setiap blok dilakukan pengurangan nilai pixel sebesar 128 dengan tujuan untuk mengubah rentang nilai pixel menjadi -128...127

```

shifted = block.astype(np.float32) - 128.0

```

3. DCT



Masing-masing blok kemudian diproses Forward DCT dan menghasilkan koefisien frekuensi. Koefisien kiri atas berisi frekuensi rendah, sedangkan semakin ke kanan bawah makin tinggi frekuensinya. Untuk embed watermark menggunakan koefisien tengah (4,4) karena area ini masih tidak terlalu terlihat oleh mata tetapi masih lebih tahan dibanding frekuensi yang lebih tinggi.

```
return cv2.dct(shifted)
```

4. Embed Watermark

Bit dari watermark tadi ditanamkan ke dalam tiap blok dari gambar asli. 1 bit watermark mewakili 1 blok (8x8 pixel). Logically, apabila bitnya bernilai 1 maka indeks kuantisasi koefisien $C[4,4]$ dibuat ganjil. Apabila bitnya 0 maka indeks kuantisasi koefisien $C[4,4]$ dibuat genap.

```
current_index = int(np.round(dct_block[u, v] / quant_step))

target_index                                     =
self._nearest_index_with_parity(current_index, int(bit))

edited[u, v] = target_index * quant_step
```

Koefisien (4,4) dipilih karena koefisien ini berada di area mid-frequency. Jika memakai frekuensi rendah seperti (0,0), perubahan akan mudah terlihat oleh mata manusia. Jika memakai frekuensi tinggi misal (7,7), watermark akan terlalu mudah hilang saat quantization.

5. Quantization

Setelah watermark ditanam di DCT, blok masuk ke proses quantization

```
quantized = self._quantize(dct_block)
```

dengan fungsi quantizationnya

```
return np.round(dct_block / self.quantization_matrix)
```

Untuk quantization matrix sendiri dibuat dari standar JPEG luminance matrix

```
STANDARD_LUMINANCE_Q = np.array([...], dtype=np.float32)
```

Kemudian blok selanjutnya diskalakan berdasarkan quality_factor

```
if quality_factor < 50:
    scale = 5000 / quality_factor
else:
    scale = 200 - (2 * quality_factor)
scaled = np.floor((cls.STANDARD_LUMINANCE_Q * scale + 50) /
100)
```

Artinya apabila QF nya tinggi, matriks kuantisasinya semakin kecil, yang berarti detail detail gambar asli akan lebih banyak yang bertahan. Apabila QF nya rendah, maka matriks kuantisasinya akan lebih tinggi, dan banyak koefisien yang hilang/dibulatkan. Karena watermark disimpan sebagai paritas indeks kuantisasi, QF rendah atau kompresi ulang JPEG dapat mengubah nilai indeks koefisien sehingga paritasnya berubah. Jika indeks yang awalnya ganjil berubah menjadi genap, bit 1 dapat terbaca sebagai 0, dan sebaliknya. Selain itu, quantization juga menentukan bin/index tempat bit disimpan. Sehingga QF rendah tetap bisa merusak karena rounding, clipping, dan JPEG recompression dapat mengubah parity indeks.

6. Dequantization

Setelah quantization, koefisien masih dalam bentuk nilai yang terkuantisasi. Agar bisa di inverse DCT, maka skalanya harus dikembalikan dulu

```
dequantized = quantized * self.quantization_matrix
```

Contoh:

```
DCT asli = 83
```

```
Q = 16
```

```
quantized = round(83 / 16) = 5
```

```
dequantized = 5 * 16 = 80
```

Nilai akhirnya tidak kembali sama persis ke 83, karena informasi hilang saat dirounding.

7. Inverse DCT dan Inverse Shift

Setelah dequantization, blok dikembalikan dari domain frekuensi ke domain spasial

```
spatial = cv2.idct(dequantized.astype(np.float32)) + 128.0
```

cv2.idct(...) menghasilkan blok pixel kembali, tetapi masih dalam kondisi level-shifted. Karena di awal dikurangi 128, pada tahap ini nilai 128 ditambahkan kembali. Setelah itu nilai pixel dibatasi pada rentang 0-255 agar valid sebagai gambar

```
return np.clip(np.round(spatial), 0, 255).astype(np.uint8)
```

8. watermarked.jpg

Setelah semua blok selesai diproses, channel Y yang sudah ada watermarknya digabungkan kembali dengan channel warna asli

```
ycrcb = np.dstack((luminance, chroma)).astype(np.uint8)
return cv2.cvtColor(ycrcb, cv2.COLOR_YCrCb2BGR)
```

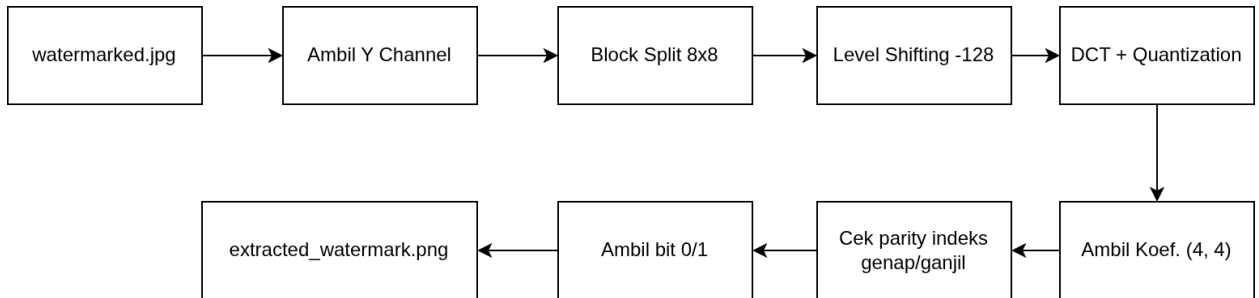
Kemudian gambar hasil disimpan

```
if not cv2.imwrite(output_path, watermarked): raise
IOError(f"Could not write watermarked image: {output_path}")
```

```
watermarked_image_path = "watermarked.jpg"
```

Hasil akhir dari semua proses ini adalah gambar yang secara visual mirip dengan input.jpg, tetapi koefisien DCT mid-frequency pada blok-blok tertentu sudah membawa bit dari watermark.png melalui paritas indeks kuantisasi koefisien tersebut.

b. Penjelasan Proses Ekstraksi Watermark



1. watermarked.jpg

Pada tahap ekstraksi, program akan membaca gambar hasil watermark

```
watermarked = cv2.imread(watermarked_path, cv2.IMREAD_COLOR)
```

Kemudian gambar tersebut dikirim ke fungsi extraction

```
extracted = self.extract_binary_watermark( watermarked,
watermark_shape=watermark_shape )
```

2. Ambil Y Channel

Program mengambil channel luminance Y dari gambar watermarked

```
watermarked_y, _ =
self._prepare_luminance(watermarked_image)
```

Jika gambar berwarna, prosesnya

```
ycrcb = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb) return
ycrcb[:, :, 0].astype(np.uint8), ycrcb[:, :, 1:].copy()
```

Channel Y digunakan dalam ekstraksi ini karena saat embedding watermark juga ditanam pada channel luminance.

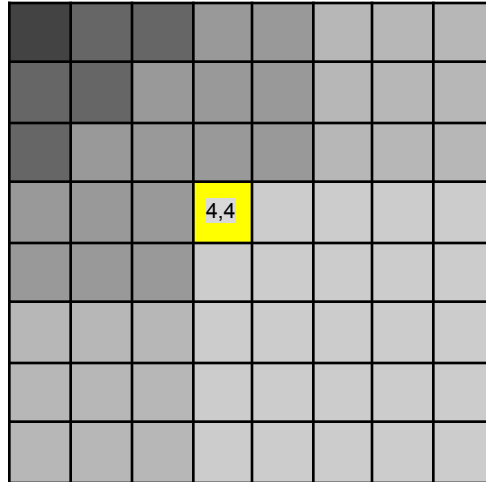
3. Block Split 8x8

Gambar watermarked akan diproses per blok 8x8

```
for row in range(0, watermarked_y.shape[0], 8):
    for col in range(0, watermarked_y.shape[1], 8):
        watermarked_block = watermarked_y[row : row + 8,
col : col + 8]
```

Satu blok 8x8 menyimpan satu bit watermark. Jika watermark berukuran 32x32, maka program membaca 1024 bit.

4. DCT + Quantization



Setiap blok kemudian diproses dengan DCT dan quantization

```
watermarked_q =  
self._quantize(self._fdct(watermarked_block))
```

Di dalam `_fdct()`, dilakukan level shifting dan DCT

```
shifted = block.astype(np.float32) - 128.0  
  
return cv2.dct(shifted)
```

Lalu quantization dilakukan dengan

```
return np.round(dct_block / self.quantization_matrix)
```

Dengan tujuan membawa blok pixel kembali ke domain yang sama dengan saat embedding, yaitu domain koefisien DCT yang terkuantisasi.

5. Ambil Koefisien (4,4)

Setelah quantization, program akan mengambil indeks koefisien di posisi (4,4)

```
coefficient_index = int(watermarked_q[u, v])
```

Nilai u, v nya berasal dari konfigurasi

```
coefficient=(4, 4)
```

Pada metode blind extraction, watermark tidak dibaca dengan membandingkan gambar asli dan gambar watermarked. Bit dibaca langsung dari nilai koefisien watermarked itu sendiri.

6. Ambil Bit 0/1

Bit watermark diambil dari parity koefisien tersebut

```
extracted_bits.append(self._parity(coefficient_index))
```

Untuk fungsi parity yang digunakan yaitu

```
return abs(value) % 2
```

Logicnya, apabila koefisien adalah genap, maka bitnya 0. Sedangkan apabila koefisiennya ganjil maka bitnya 1. Contoh:

```
coefficient_index = 8
```

```
8 % 2 = 0
```

```
bit = 0
```

```
-----
```

```
coefficient_index = 9
```

```
9 % 2 = 1
```

```
bit = 1
```

7. extracted_watermark.png

Setelah jumlah bit yang dibaca sudah sesuai ukuran watermark, bit-bit tersebut disusun kembali

```
extracted = np.array(extracted_bits, dtype=np.uint8)
```

```
return (extracted.reshape(watermark_shape) *  
255).astype(np.uint8)
```

Karena bit yang diambil hanya 0 dan 1, maka selanjutnya dikalikan 255 agar menjadi gambar hitam/putih. Kemudian disimpan sebagai


```
cv2.imwrite(output_path, extracted)
```

Output akhirnya adalah `extracted_watermark.png`

c. Demo Pengujian

Bagian ini mendemonstrasikan proses eksekusi dari sistem *watermarking* yang telah dibangun, meliputi proses penyisipan (*embedding*), ekstraksi (*extraction*), serta pengujian ketahanan (*robustness*) *watermark* terhadap berbagai tingkat kompresi JPEG.

1. Skenario dan Data Pengujian

Pengujian dilakukan menggunakan dua berkas citra utama sebagai input:



`input.jpg`



`watermark.png`

Citra Asli: `input.jpg` (Foto berwarna saya)

Citra Watermark: `watermark.png` (Citra hitam-putih berukuran 32x32 pixel)

Parameter standar yang digunakan pada pengujian awal adalah koordinat penyisipan pada *mid-frequency* koefisien (4,4) dengan *Embedding Quality Factor* (QF) sebesar 90.

2. Demonstrasi Penyisipan (Embedding)

Proses penyisipan dilakukan dengan menjalankan skrip *embedding*. Sistem membaca bit *watermark*, mengubah citra host ke domain frekuensi (DCT), memodifikasi paritas pada indeks kuantisasi, dan mengembalikannya ke domain spasial.

Hasil Visualisasi:



input.jpg



watermarked.jpg

Secara kasatmata manusia, tidak terdapat perbedaan atau degradasi kualitas yang signifikan antara citra asli dan citra *watermarked*. Perubahan nilai piksel terjadi karena program memodifikasi koefisien DCT agar indeks kuantisasinya memiliki paritas sesuai bit watermark.

3. Demonstrasi Ekstraksi (Extraction)

Proses ekstraksi bersifat *blind extraction*, di mana sistem hanya menerima *watermarked.jpg* tanpa memerlukan *input.jpg*. Sistem mengekstraksi bit 0 dan 1 berdasarkan paritas koefisien kuantisasi (genap 0, ganjil 1) di titik (4,4) pada setiap blok 8x8. Pada proses ini parameter `embedding_qf` harus sama dengan QF yang digunakan saat embedding, karena nilai tersebut menentukan quantization matrix untuk membaca kembali parity koefisien DCT.

Hasil Visualisasi:



watermark.png



extracted_watermark.png

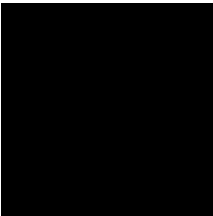
Pada kondisi ideal, sistem berhasil merekonstruksi *watermark* secara utuh. Susunan piksel hitam dan putih pada *extracted_watermark.png* identik dengan *watermark.png*.

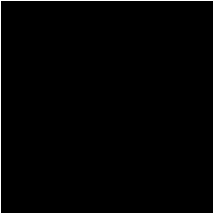
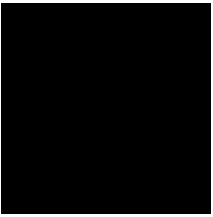
4. Pengujian Ketahanan (Robustness) terhadap Kompresi JPEG

Pengujian ketahanan dilakukan untuk mengetahui pada nilai Quality Factor berapa watermark mulai gagal diekstraksi. Pada tahap ini, watermark terlebih dahulu ditanam dengan `embedding_qf = 90`. Setelah menghasilkan `watermarked.jpg`, gambar tersebut dikompresi ulang menggunakan beberapa nilai `recompress_qf`. Perbedaanannya `embedding_qf` adalah QF saat watermark disisipkan, sedangkan `recompress_qf` adalah QF saat `watermarked.jpg` dikompresi ulang. Kompresi ulang ini dianggap sebagai serangan JPEG recompression. Semakin kecil nilai `recompress_qf`, semakin besar kompresi yang diberikan, sehingga semakin banyak informasi frekuensi yang dapat berubah atau hilang.

Pada setiap nilai `recompress_qf`, hasil ekstraksi dibandingkan secara visual dengan watermark asli. Jika bentuk watermark masih jelas, maka watermark dianggap masih berhasil diekstraksi. Jika bentuk watermark rusak atau tidak lagi menyerupai watermark asli, maka watermark dianggap gagal diekstraksi.

5. Hasil Pengujian

recompress QF	Hasil Ekstraksi	Extraction Accuracy	Keterangan
90		1.0	Hasil extract watermark masih utuh
85		1.0	Hasil extract watermark juga masih tetap utuh
80		0.1318	Hasil extract watermark sudah tidak dapat dikenali

50		0.1318	Hasil extract watermark sudah tidak dapat dikenali
30		0.1318	Hasil extract watermark sudah tidak dapat dikenali